

A Modular ZK-Rollup Prototype for Benchmarking Layer-2 Scalability Bottlenecks

Sunimal Rathnayake¹, Fernando T.H.L.², Gamage M.S.³, Lishan S.D.⁴,
Perera M.V.D.P.D.S.⁵

^{1,2,3,4,5}Department of Computer Science & Engineering, University of Moratuwa, Sri Lanka.

¹sunimal@cse.mrt.ac.lk, ²lakshara.21@cse.mrt.ac.lk, ³malindug.21@cse.mrt.ac.lk, ⁴lishan.21@cse.mrt.ac.lk, ⁵shalindra.21@cse.mrt.ac.lk

Abstract—Ethereum’s limited Layer-1 throughput has motivated the adoption of Layer-2 rollups, where ZK-Rollups provide strong security and fast finality through validity proofs. However, practical ZK-Rollup performance depends heavily on configuration choices such as batch size, sequencing policy, data availability mode, and proof-generation overhead. Existing production rollups are difficult to modify for controlled experimentation, limiting reproducible analysis of these trade-offs.

This paper presents a modular ZK-Rollup prototype designed for benchmark-driven performance evaluation. The prototype separates the sequencer, executor, submitter, and prover components, enabling controlled experiments over different batching, scheduling, and data availability configurations. The prototype is evaluated using synthetic workloads, and key metrics such as throughput, latency, gas cost per transaction, batch finalization time, and prover-side overhead are analyzed.

The proposed modular evaluation framework isolates key bottlenecks in the rollup pipeline, exposing how configuration choices shift the throughput-latency-cost trade-off. By serving as a reproducible basis for studying ZK-Rollup scalability, the framework offers practical guidance for selecting optimal architectural parameters.

Index Terms—ZK-Rollups, Ethereum, Layer-2 Scaling, Blockchain Scalability, Benchmarking, Sequencer, Data Availability

I. INTRODUCTION

Ethereum Layer-1 provides strong security and decentralization but remains constrained by limited throughput, congestion, and fee volatility [1], [2]. This limitation reflects the blockchain trilemma, where decentralized blockchain systems must balance scalability, security, and decentralization rather than maximizing all three simultaneously [3]. Ethereum’s current scaling direction therefore relies heavily on Layer-2 rollups, which move transaction execution off-chain while using Layer-1 for settlement, verification, and data availability [4], [5].

Within this rollup-centric design space, ZK-Rollups are particularly attractive because they submit cryptographic validity proofs that allow Layer-1 contracts to verify off-chain state transitions without re-executing every transaction. This preserves the base layer’s security model while reducing the amount of computation performed directly on Layer-1.

However, practical ZK-Rollup performance depends heavily on interacting bottleneck factors, including proof-generation overhead, sequencer scheduling policies, data availability (DA) mode costs, and batch size/frequency trade-offs [6], [7]. Existing production ZK-Rollups are optimized for deployment and

user adoption, but their monolithic complexity makes them difficult to modify for controlled academic benchmarking.

To address this gap, this paper presents a modular ZK-Rollup research prototype designed to separate sequencing, execution, proving, submission, and data availability concerns. By doing so, it supports reproducible, configuration-level benchmarking. The system is designed as a controlled experimental environment and is not a production-ready system or a full zkEVM.

The main contributions of this work are:

- We design and implement a modular ZK-Rollup research prototype with separated sequencer, executor, prover, submitter, smart contract, and DA components.
- We expose configurable batching, sequencing, and DA policies to support controlled benchmarking.
- We define a reproducible methodology for evaluating throughput, latency, gas per transaction, proof or mock-proof time, finalization time, DA footprint, and queueing behavior.
- We analyze the resulting throughput-latency-cost trade-offs and identify bottlenecks that guide future ZK-Rollup optimization.

II. BACKGROUND AND RELATED WORK

A. Ethereum Scalability and Rollups

Ethereum’s rollup-centric roadmap relies on Layer-2 solutions to move execution off-chain while utilizing Layer-1 for settlement and data availability [4]. ZK-Rollups use validity proof systems (such as Groth16 or PLONK) to mathematically guarantee off-chain execution correctness [8], [9].

B. Performance Bottlenecks and Benchmarking

Recent benchmarking efforts highlight multiple bottlenecks in ZK-Rollups. Generating validity proofs remains a significant computational hurdle [6]. Furthermore, data availability poses a trade-off between cost and security; the introduction of EIP-4844 (proto-danksharding) aims to alleviate this by providing cheaper blob-based storage [10]–[12].

EIP-4844 introduces blob-carrying transactions where large rollup data are made available through blob space rather than permanent calldata. The EVM cannot directly access blob contents, but it can access commitments to blob data, making blobs suitable for rollup data availability rather than contract execution data. Ethereum’s data availability documentation

TABLE I
POSITIONING OF THE PROPOSED PROTOTYPE AGAINST PRODUCTION ROLLUPS

System	Main Goal	Configurable Internals	Benchmark Focus
zkSync Era	Production L2	Limited	Public deployment metrics
Polygon zkEVM	EVM compatibility	Limited	zkEVM execution and proving
Starknet	Production validity rollup	Limited	Cairo-based execution
Proposed prototype	Research benchmarking	High	Batch, sequencing, DA, proof delay

Modular ZK-Rollup Prototype Architecture

Users → Sequencer → Executor → Prover → Submitter → L1 Contract

Fig. 1. Architecture of the proposed modular ZK-Rollup benchmarking prototype.

also notes that after EIP-4844, some rollups post data to cheaper blob storage rather than permanent calldata, but blob data are only guaranteed for a fixed window [5]. This changes the cost structure of rollup submission and motivates evaluating calldata and blob-based DA separately.

The design of the sequencer and the chosen transaction ordering policies also heavily influence latency and fairness [13]. Existing studies often analyze public datasets or production traces [7], [14].

C. Research Gap

While prior work examines rollup architectures, costs, data availability, sequencer decentralization, and production traces, there is still a need for a compact modular framework that supports controlled experiments over batching, sequencing, DA, and proof overhead under identical workloads.

Unlike production rollups, the proposed prototype is not optimized for broad application compatibility. Its purpose is to expose internal rollup controls for repeatable experiments. This makes it complementary to production-trace studies and public datasets rather than a replacement for them.

III. SYSTEM ARCHITECTURE

A. Overview

The proposed framework is designed as a modular ZK-Rollup research prototype. The architecture deliberately separates policy decisions from execution and settlement mechanisms. This design prioritizes observability and reproducibility over production completeness, avoiding the complexity of a full production zkEVM.

TABLE II
PROTOTYPE COMPONENTS AND EXPERIMENTAL CONTROLS

Component	Role	Configurable Control
Sequencer	Orders and batches transactions	FCFS, fee-priority, time-boost
Executor	Applies state transitions	State tree / batch execution
Prover	Models proof overhead	Mock delay / proof mode
Submitter	Commits batches to L1	Calldata, blob, off-chain DA
L1 Contract	Verifies commitments	Verifier and DA provider abstraction

B. Transaction Lifecycle and Components

The end-to-end transaction lifecycle involves several decoupled components, as illustrated in Fig. 1.

Workload Generator: Produces synthetic transaction traffic at controlled rates, supporting various workload regimes such as balanced steady-state or bursty arrivals.

Sequencer: Maintains a transaction pool and orders transactions based on configurable scheduling policies (e.g., FCFS, fee-priority, time-boost, or fair BFT). It includes a batch orchestrator that determines when to form a new batch based on fixed-size, fixed-interval, or soft-cap rules. It also monitors a forced transaction queue to simulate censorship resistance.

Executor: The state manager applies state transitions to a sparse Merkle tree. It validates nonces and balances, separating execution from transaction ordering.

Prover: The mock-prover mode parameterizes proof delay, allowing the proving stage to be studied without requiring full circuit execution in every benchmark run.

Submitter and Layer-1 Contract: The submitter handles bit-packing and compression strategies to commit data to Layer-1. It supports abstractions for different DA providers, routing data as calldata, blobs, or off-chain DA. The L1 smart contract acts as the verifier abstraction.

Telemetry: Granular data collection is embedded throughout to monitor queueing delays, prover utilization, and submission costs.

C. Design Rationale

The prototype deliberately prioritizes modularity and observability over production completeness. Unlike production ZK-Rollups, where batching, proving, data availability, and submission logic are tightly optimized for deployment, the proposed framework exposes these mechanisms as configurable experimental controls. This allows the same workload to be replayed across different batch policies, sequencing policies, and data availability modes while keeping the remaining pipeline fixed. This design makes it possible to attribute performance changes to specific architectural choices rather than to unrelated implementation differences.

The separation between the sequencer, executor, prover, submitter, and Layer-1 contract also supports controlled bottleneck isolation. For example, the prover can be executed in mock mode to parameterize proof delay, while the submitter can switch between calldata, blob-based DA, and off-chain DA to isolate data-posting overhead. This makes the prototype suitable for studying the throughput–latency–cost frontier of

TABLE III
EXPERIMENTAL CONFIGURATIONS USED FOR PROTOTYPE EVALUATION

Config	Batch Policy	Seq. Policy	DA Mode	Workload	Prover Mode
C1	Fixed-size	FCFS	Calldata	Steady-state	Mock
C2	Fixed-size	Fee-priority	Calldata	Steady-state	Mock
C3	Soft-cap	Time-boost	Calldata	Bursty	Mock
C4	Soft-cap	FCFS	Blob	Bursty	Mock
C5	Tx-aware	Fair BFT	Blob	Skewed	Mock
C6	Fixed-interval	Fee-priority	Off-chain	Steady-state	Mock

ZK-Rollup systems, even though it is not intended to provide full zkEVM compatibility.

IV. EXPERIMENTAL METHODOLOGY

A. Study Objective and Scope

The objective is to systematically evaluate ZK-Rollup performance trade-offs using controlled configuration sweeps. The scope assumes a simplified operation set via synthetic token transfers, measuring relative performance shifts rather than absolute production metrics.

B. Experimental Factors and Regimes

The framework exposes configurable parameters (Table III), including batching policies (fixed-size, fixed-interval, soft-cap, transaction-aware), sequencing policies, and DA modes. Workload regimes cover balanced steady-state, heavy-transaction skew, and bursty arrivals with latency SLAs.

C. Batching Policy Space

The benchmark considers three categories of batching policies. First, baseline agnostic policies close batches using either a fixed transaction count or a fixed time interval. These policies are simple and predictable but do not adapt to workload heterogeneity. Second, soft-cap policies close a batch when a resource limit is reached, such as DA size, proving budget, gas limit, or maximum waiting time. This reflects how practical rollup systems often avoid exceeding execution or posting constraints. Third, transaction-aware policies attempt to construct batches based on transaction cost profiles, for example by limiting the ratio of heavy transactions or by treating batching as a multidimensional packing problem over proof cost and DA footprint.

This policy space is intended to evaluate whether simple fixed batching remains efficient under heterogeneous workloads, or whether transaction-aware policies become necessary when proof cost and DA footprint vary across transaction types.

D. Workload Model

Synthetic transactions are grouped into three abstract cost classes. Light transactions represent simple transfers with low proving cost and low DA footprint. Medium transactions represent ERC-20 style transfers or simple swaps with moderate proving and DA cost. Heavy transactions represent more complex contract interactions with higher proving cost and stricter latency impact. The workload generator varies the proportion of these classes across steady-state, skewed, and

Benchmark Execution Pipeline

Workload → Prototype Components → Telemetry Aggregation → Metrics

Fig. 2. Benchmark execution pipeline used for controlled configuration sweeps.

TABLE IV
SUMMARY OF PRELIMINARY EXPERIMENTAL RESULTS

Config	Comm. TPS	Avg. Lat.	P95 Lat.	Cost/Tx	Proof Time	DA Bytes
C1	357.1	2.1s	4.5s	1.2M	0.5s	120KB
C2	342.5	1.8s	3.9s	1.3M	0.5s	120KB
C3	280.4	3.5s	7.2s	1.1M	0.8s	95KB
C4	290.8	3.2s	6.8s	0.4M	0.8s	98KB
C5	250.2	4.1s	8.5s	0.3M	1.2s	85KB
C6	310.6	2.5s	5.1s	0.1M	0.6s	0KB

bursty regimes. This allows the evaluation to study not only average throughput but also how transaction heterogeneity affects queueing delay, prover utilization, and SLA violations.

E. Cost Model and Metrics

The total transaction fee is modeled as the sum of the L2 fee (execution/proving resource usage) and the L1 fee (blob or calldata cost, commitment cost, verification cost).

The evaluation collects the following metrics:

- Submitted and Committed TPS
- Average and P95/P99 Latency
- Cost and Gas per Transaction
- Proof/Mock-Proof and Finalization Time
- DA Bytes and Queueing Delay
- Prover Utilization and Expired Transactions

F. Procedure and Hypotheses

The procedure involves initializing the prototype, applying the workload generator, and aggregating telemetry (Fig. 2). The experiments test hypotheses such as: agnostic batching degrades with heterogeneity, soft-cap improves robustness but may waste capacity, and transaction-aware batching dominates after a heterogeneity threshold.

V. RESULTS AND DISCUSSION

A. Evaluation Results

The evaluation framework reports multi-dimensional metrics across the configured experimental space. Table IV illustrates the preliminary benchmark outputs under synthetic workloads.

All preliminary measurements were obtained from controlled synthetic workloads using the same benchmark harness across configurations. The values should be interpreted as relative comparisons between configurations rather than claims of production mainnet performance. Since the prototype uses a mock-prover mode, proof time represents parameterized proving delay rather than full circuit execution time.

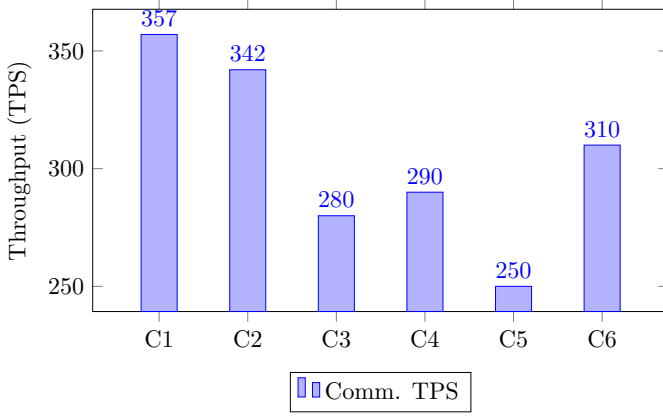


Fig. 3. Throughput comparison across the evaluated experimental configurations.

B. Throughput, Latency, and Cost Trade-offs

Analysis of the results indicates that increasing batch size amortizes Layer-1 submission costs, decreasing the cost per transaction. However, this increases queuing delay, demonstrating a strict latency-cost trade-off. For instance, comparing C1 with C3, the higher throughput in C1 (357.1 TPS) comes with higher DA costs (120KB) and shorter latency, while C3 trades off throughput (280.4 TPS) for lower DA footprint (95KB). Data availability choices further impact costs; blob-based DA (C4, C5) drastically reduces the L1 footprint cost compared to standard calldata (C1, C2).

C. Design Implications

These interactions highlight several implications for rollup designers. TPS alone is insufficient to capture rollup performance; it must be contextualized with latency and cost.

- Batch sizes should be chosen based on the application’s latency tolerance.
- Transaction-aware batching is beneficial when workloads are heterogeneous.
- Prover utilization must be monitored as a first-class metric, as proof-generation can easily become the primary bottleneck.
- The choice of DA mode should align with specific security and cost assumptions.
- Soft finality (L2 confirmation) should be distinctly separated from L1 finality in reporting.

VI. CONCLUSION AND FUTURE WORK

This paper presented a modular ZK-Rollup research prototype designed to enable controlled experimental measurement of Layer-2 scalability bottlenecks. By isolating the sequencer, executor, prover, submitter, and data availability components, the framework supports reproducible configuration-level benchmarking. The defined methodology and preliminary evaluation highlight the complex trade-offs between throughput, latency, gas cost, and proof overhead.

A. Limitations

The proposed prototype is a research tool, not a production-ready system or a full zkEVM. The preliminary evaluation currently relies on synthetic workloads and a simplified operation set within a local testnet environment. Furthermore, the use of a mock prover simulates computational delay but does not capture the full cryptographic complexity of real circuit execution. In addition, the real benchmark values and synthetic measurements presented here may not represent complex DeFi behavior, and local or testnet measurements could differ from mainnet deployment.

B. Future Work

Future work will focus on integrating full cryptographic prover capabilities and proof aggregation to measure exact hardware overheads. The framework will be extended to evaluate real trace-based workloads, support a complete trustless deposit and withdrawal path, and experiment with decentralized sequencers and stronger DA backends (e.g., Celestia or Avail). Ultimately, a public reproducibility package will be released to support the broader research community.

REFERENCES

- [1] V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform,” <https://ethereum.org/en/whitepaper/>, 2014.
- [2] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” Ethereum Project, Tech. Rep. 151, Apr 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [3] Y. Fu, M. Jing, J. Zhou, P. Wu, Y. Wang, L. Zhang, and C. Hu, “Quantifying the blockchain trilemma: A comparative analysis of algorand, ethereum 2.0, and beyond,” *Proceedings of the IEEE*, vol. 112, no. 2, pp. 158–174, 2024.
- [4] Ethereum Foundation, “Scaling ethereum,” <https://ethereum.org/en/layer-2/>, 2024, accessed: 2026-05-02.
- [5] —, “Data availability,” <https://ethereum.org/developers/docs/data-availability/>, 2026, accessed: 2026-05-02.
- [6] M. A. Habib, “Analyzing performance bottlenecks in zero-knowledge proof based rollups on ethereum,” *arXiv preprint arXiv:2503.22709*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.22709>
- [7] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, and B. Livshits, “Analyzing and benchmarking zk-rollups,” *Cryptology ePrint Archive*, Paper 2024/889, 2024. [Online]. Available: [url{https://eprint.iacr.org/2024/889}](https://eprint.iacr.org/2024/889)
- [8] J. Groth, “On the size of pairing-based non-interactive arguments,” in *EUROCRYPT 2016*, 2016, pp. 305–326.
- [9] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” <https://eprint.iacr.org/2018/046.pdf>, 2018.
- [10] V. Buterin *et al.*, “Eip-4844: Shard blob transactions,” <https://eips.ethereum.org/EIPS/eip-4844>, 2022, accessed: 2026-05-02.
- [11] M. B. Saif, S. Migliorini, and F. Spoto, “A survey on data availability in layer 2 blockchain rollups: Open challenges and future improvements,” *Future Internet*, vol. 16, no. 9, p. 315, 2024. [Online]. Available: <https://doi.org/10.3390/fi16090315>
- [12] T. Lavaur, J. Detchart, J. Lacan, and C. P. C. Chanel, “Modular zk-rollup on-demand,” iSAE-SUPAERO, University of Toulouse, 2023.
- [13] S. Motepalli, L. Freitas, and B. Livshits, “Sok: Decentralized sequencers for rollups,” *arXiv preprint arXiv:2310.03616*, 2023.
- [14] M. I. Silva, J. Messias, and B. Livshits, “A public dataset for the zksync rollup,” *arXiv preprint arXiv:2407.18699*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.18699>